



RETHINKING • HOW • WORK • WORKS

Agentic AI Engineering Internship , 2026

WEEK 1

AI Fundamentals & Agentic AI Foundations

Monday 22 June – Friday 26 June 2026

From zero to agent , understanding the modern AI landscape

Week Overview

Week 1 establishes the conceptual and practical foundation for the entire internship. Interns move from understanding how large language models work to building their first working agentic system using LangChain and Groq.

Detail	Information
Week Dates	Monday 22 June – Friday 26 June 2026
Theme	AI Fundamentals & Agentic AI Foundations
Primary Stack	Python · LangChain · Groq API · ChromaDB (in-memory)
Total Commitment	20 hours (4 hours/day · 5 days)
Deliverable Due Friday	1 Intermediate Project + 1 Production Project (choose one of each)

Learning Objectives

Understand how large language models work at a conceptual and practical level

Distinguish between AI, ML, deep learning, and LLMs

Explain what makes an AI 'agentic' versus reactive

Build a simple conversational agent using LangChain and Groq

Understand tokens, context windows, temperature, and sampling parameters

Write effective system prompts for agent personas

Topics & Subtopics

Topic	Subtopics
LLM Foundations	Transformers architecture, tokenization, attention, training vs inference, RLHF, instruction tuning
Agentic AI Concepts	Reactive vs proactive agents, planning, tool use, memory, the ReAct pattern, agent loops
LangChain Foundations	LLMs, ChatModels, PromptTemplates, Chains, LCEL, output parsers, runnables
Groq API	Available models (llama-3, mixtral, gemma), speed benchmarks, pricing, rate limits
Prompt Engineering Intro	System prompts, few-shot prompting, chain-of-thought, instruction clarity
Python for AI	Async functions, type hints with Pydantic, error handling, logging best practices

Daily Breakdown

Each day is structured around a core learning block followed by applied, hands-on practice. Complete each day's practice before moving to the next , concepts compound throughout the week.

Day	Focus	Core Learning	Applied Practice
Mon	LLM Foundations	Watch Karpathy 'Intro to LLMs'. Read Attention Is All You Need summary. Study transformer architecture diagrams.	Build first Groq API call in Python. Explore different models (llama-3-8b, mixtral-8x7b). Experiment with temperature (0 vs 1 vs 2).
Tue	Agentic AI Concepts	Read 'ReAct: Synergizing Reasoning and Acting' paper summary. Study the Agent Loop: Perceive → Plan → Act → Observe.	Build a simple ReAct-style loop manually in Python (no framework). Agent decides which of 3 tools to call based on user input.
Wed	LangChain Core	LangChain LCEL tutorial. Study: Runnable, RunnablePassthrough, RunnableParallel. Build 3 different chain patterns.	Build a document Q&A chain (load text file → split → embed → retrieve → answer). Use ChromaDB in-memory.
Thu	Prompt Engineering	Study: system prompts, zero-shot vs few-shot, chain-of-thought. Read Anthropic Prompt Engineering guide.	Lab: Build a persona-based agent (customer support bot, code reviewer, data analyst). Measure output quality differences.
Fri	Integration + Demo	Integrate all concepts: multi-turn chatbot with memory, custom persona, Groq backend, rich terminal UI.	Weekly standup prep. Demo your chatbot. Peer code review. Start intermediate project.

Recommended Resources

These resources are drawn from official documentation and original research. Complete the highlighted readings before their corresponding daily session.

Type	Resource	Why It Matters
Video	Andrej Karpathy – Intro to Large Language Models (YouTube)	Best conceptual intro to LLMs for engineers
Video	Andrej Karpathy – Let's Build GPT From Scratch (YouTube)	Deep dive into transformer internals
Paper	Attention Is All You Need (Vaswani et al., 2017)	Foundational transformer paper
Paper	ReAct: Synergizing Reasoning and Acting in LLMs	Foundation of agentic AI patterns
Docs	LangChain Python Docs – Get Started	Official LangChain reference
Docs	Groq Cloud Documentation	API reference and model details
Blog	Anthropic – What Is Constitutional AI?	Understanding safe agent design
Blog	LangChain Blog – LCEL Explained	Modern chain composition
Repo	LangChain Cookbook (GitHub)	100+ practical examples
Course	DeepLearning.AI – LangChain for LLM Application Development	Free, Andrew Ng-produced course

Hands-on Labs

Complete all three labs over the course of the week. Each lab builds toward the skills required for your Friday project submission.

Lab 1.1: Your First Groq Agent

Build a Python CLI chatbot that: uses Groq llama-3-8b, maintains conversation history, supports /clear and /exit commands, and shows token usage per response.

Lab 1.2: Manual ReAct Loop

Without using any framework, implement a ReAct agent that can: search a mock 'database' of facts, calculate math expressions using eval(), and decide which tool to call based on the user's question.

Lab 1.3: Prompt Engineering A/B Test

Write five different system prompts for the same task (summarising news articles). Measure accuracy, conciseness, and tone. Document your findings in a brief report.

Weekly Assessment

Complete this self-assessment before Friday's standup. These questions test conceptual clarity, not memorisation.

Question Type	Question
Conceptual	Explain the difference between a language model and an agent. What capabilities does an agent add?
Conceptual	What is the 'context window' and why does it matter for agentic systems?
Conceptual	Describe the ReAct pattern. When would you use it versus a simple chain?
Technical	What is LCEL in LangChain? Write a 5-line example of a chain using the pipe operator.
Technical	Explain the role of temperature in LLM sampling. When would you set it to 0?
Design	Design a simple agent architecture for a customer support chatbot. What tools would it need?

Weekly Standup Requirements

Every Friday standup follows the same structure. Prepare each item in advance.

Item	Detail
Demo	Live demo of your chatbot (2 minutes). Show conversation history working.
Technical Question	Explain one concept from the week that surprised you or changed how you think.
Architecture Review	Draw (on whiteboard or shared screen) the agent loop your chatbot follows.
Code Review	Share your most interesting code snippet from the week. What did you learn?
Blockers	Name one technical challenge you faced. How did you solve it (or plan to)?

Week 1 Projects

Choose exactly one Intermediate project and one Production project to complete by Friday. Both are submitted via GitHub and demonstrated in the Friday standup.

Category 1 Projects , Choose One

Project 1-I-A: Intelligent CLI Assistant

Intelligent CLI Assistant	
Difficulty	Intermediate
Description	Build a terminal-based AI assistant powered by Groq that can answer questions about a chosen topic domain (cooking, history, programming). Supports conversation history, topic switching, and formatted output using the Rich library.
Architecture	User CLI → LangChain ChatModel (Groq) → Conversation Buffer Memory → Rich Console Output. Single file Python application.
Skills Learned	LangChain chains, conversation memory, Groq API, Rich terminal UI, Python CLI design
Deliverables	GitHub repo with README, requirements.txt, demo GIF, and 3 example conversations documented
Evaluation Criteria	Conversation history works across 10+ turns. Handles errors gracefully. README clearly documents setup. Code is clean and typed.

Project 1-I-B: Prompt Engineering Evaluator

Prompt Engineering Evaluator	
Difficulty	Intermediate
Description	Build a tool that takes a task description, generates 3 different prompts automatically using an LLM, runs all 3 against test cases, and scores them on accuracy, brevity, and helpfulness.

Architecture	Task Input → Prompt Generator Agent (Groq) → 3 parallel LLM calls → Evaluator Agent → Comparison Report (JSON + console)
Skills Learned	Parallel LLM calls, structured output with Pydantic, evaluation design, LangChain LCEL
Deliverables	Working CLI tool, 5 evaluated prompt pairs, findings report in README
Evaluation Criteria	Evaluation is automated, consistent, and explainable. Results show meaningful differences between prompts.

Project 1-I-C: Domain Expert Agent

Domain Expert Agent	
Difficulty	Intermediate
Description	Build a specialized agent with a detailed persona (e.g., Python code reviewer, marketing copywriter, financial advisor). The agent uses structured prompts, refuses out-of-domain requests, and maintains consistent personality.
Architecture	System Prompt Layer → Input Classifier → Domain Guard → Expert Responder → Output Formatter
Skills Learned	Prompt engineering, output parsing, domain classification, agent guardrails
Deliverables	Working agent with domain guard, 10 test conversations, documented edge cases
Evaluation Criteria	Agent maintains persona consistency. Domain guard works correctly. Response quality is high.

Category 2 Projects , Choose One

Project 1-P-A: AI-Powered Customer Support Platform

AI-Powered Customer Support Platform	
Difficulty	High
Description	Build a production-ready customer support chatbot as a FastAPI REST API with endpoints for starting sessions, sending messages, and retrieving history. Includes rate limiting, input validation, logging, and a simple React frontend or Streamlit UI.
Architecture	React/Streamlit UI → FastAPI Backend → LangChain ConversationChain (Groq) → SQLite Session Storage → Logging Middleware → Pydantic Request/Response Models
Skills Learned	FastAPI, REST API design, session management, SQLite, Pydantic v2, CORS, logging, error handling
Deliverables	GitHub repo with Docker support, API documentation (auto-generated by FastAPI), Postman collection, README with architecture diagram, and live demo
Evaluation Criteria	API follows REST best practices. Docker compose works on first run. Error cases handled. 90%+ test coverage on core logic.

Project 1-P-B: Multi-Model Comparison Engine

Multi-Model Comparison Engine	
Difficulty	High
Description	Build a benchmarking tool that evaluates multiple Groq models (llama-3-8b, llama-3-70b, mixtral-8x7b, gemma-7b) on a standardized task suite. Generates statistical comparison reports with latency, quality, and cost metrics.
Architecture	Task Suite YAML → Parallel Model Runner (asyncio) → Response Collector → Statistical Analyzer → Report Generator (HTML + JSON) → GitHub Pages Deploy
Skills Learned	Async Python, statistical analysis, data visualization (matplotlib/plotly), YAML config, HTML report generation

Deliverables	Published benchmark report, automated test runner, GitHub Actions CI pipeline, documented methodology
Evaluation Criteria	Benchmarks are reproducible. Statistical analysis is valid. Report is readable by non-engineers. CI pipeline runs on push.

Project 1-P-C: Agentic Research Assistant

Agentic Research Assistant	
Difficulty	High
Description	Build a research assistant agent that can take a question, plan a research strategy, make sequential LLM calls to explore subtopics, synthesize findings, and produce a structured report with citations and confidence scores.
Architecture	Question Input → Planner Agent → Sequential Research Loop (3–5 Groq calls) → Synthesis Agent → Report Formatter → Streamlit UI with progress display
Skills Learned	Multi-step agent loops, planning patterns, output structuring with Pydantic, Streamlit UI, progress indicators
Deliverables	Streamlit app deployed to Streamlit Cloud (free), GitHub repo, sample research reports on 3 different topics, architecture diagram
Evaluation Criteria	Agent produces coherent multi-section reports. Planning is visible to user. Confidence scores are meaningful. Deploys successfully.

Week 1 Completion Checklist

Confirm every item below before Friday's standup.

Checklist Item	
☐	All 5 daily learning sessions completed (Mon–Fri)
☐	Lab 1.1 , Groq CLI chatbot built and working
☐	Lab 1.2 , Manual ReAct loop implemented without a framework
☐	Lab 1.3 , Prompt A/B test completed with documented findings
☐	Weekly Assessment questions answered in writing
☐	One Intermediate project chosen, built, and pushed to GitHub
☐	One Production project chosen, built, and pushed to GitHub
☐	Both projects have complete README files with setup instructions
☐	Architecture diagram prepared for Friday's standup review
☐	Demo rehearsed and ready to present in under 2 minutes